Sovereign On-Premise Agentic AI Workbench

Open-Weight Multimodal LLMs with Graph RAG
for Confidential Industrial Work

Problem Statement	SIH26117
Title	Sovereign On-Premise Agentic AI Workbench using Open-Weight Multimodal LLMs for Confidential Industrial Work
Organisation	Mangalore Refinery and Petrochemicals Limited (MRPL)
Category	Software
Theme	Smart Automation
Event	Smart India Hackathon 2026

Abstract. Refineries, public sector undertakings, defence-linked manufacturers and government offices generate large volumes of routine but highly sensitive knowledge work. This material cannot be sent to cloud AI assistants, so the work is either done manually or confidential content is quietly pasted into public tools. This report describes a self-hosted, air-gapped agentic AI workbench that runs entirely on an organisation's own hardware. It serves multiple open-weight models with automatic task-based selection, executes multi-step agentic work through local tools, understands scanned and handwritten documents, produces real office deliverables, and grounds every answer in the organisation's own corpus through a hybrid vector and knowledge graph retrieval layer. Critically, it does not merely assert its sovereignty: it demonstrates the absence of external network activity through kernel-level isolation, a live egress monitor and a tamper-evident audit chain.

Reference implementation verified on a single NVIDIA RTX 5080 (16 GB).

September 8, 2026

Contents

1 Executive Summary

1.1 The problem in one paragraph

An engineer at a refinery needs to read a scanned inspection report, cross-check it against the governing standard operating procedure, work out which downstream units are affected, and draft an approval note. Every artefact involved — the piping and instrumentation diagram, the vendor correspondence, the financials, the unreleased design — is confidential. Company policy forbids sending it to a cloud assistant. So the engineer either does the work by hand, losing hours, or pastes the material into a public tool anyway, creating exactly the exposure the policy exists to prevent.

1.2 What was built

A complete, working system that runs on one consumer GPU with no network connection. It comprises:

- a **model gateway** serving five open-weight models with automatic, VRAM-aware selection driven by a declarative manifest;
- an **agent** that plans multi-step work, calls local tools, critiques its own output and iterates;
- a **retrieval layer** combining dense vectors, sparse BM25 and a typed **knowledge graph**, with four retrieval strategies chosen by query shape;
- a **multimodal ingestion pipeline** handling text PDFs, scanned pages, handwriting and engineering drawings;
- a **deliverable generator** producing genuine `.docx`, `.pptx` and `.xlsx` files, not chat transcripts;
- a **sovereignty subsystem** providing kernel-level network isolation, a live egress monitor, a deliberate breach canary, and a hash-chained tamper-evident audit log.

1.3 Status at the time of writing

Measure	Value	Note
Python source	3 124 lines	46 modules across 9 packages
Automated tests	61 passing	full suite runs in under 1 second
Dependencies	99 packages	all inside a project-local virtualenv
Ontology	23 node types	31 relation types, 43 endpoint signatures
Open-weight models	5	approximately 24 GB of weights
Knowledge graph	499 nodes	391 typed relations from a 5-document corpus
Community summaries	37	enables corpus-wide thematic queries
Verified rubric points	5 of 5	each demonstrated, not asserted

1.4 The three claims that distinguish this work

1. Sovereignty is demonstrated, not claimed

The problem statement is explicit that proof — not assertion — is what counts. Generated code executes inside an unprivileged network namespace where a connection attempt to `1.1.1.1:53` returns `OSError: Network is unreachable`. This is a kernel guarantee, not a policy setting, and it was verified experimentally.

2. Graph RAG answers questions vector search structurally cannot

"If we isolate P-101A, which downstream units are affected?" requires traversal. Similarity is not connectivity. A two-hop dependency shares no vocabulary with the query, so an embedding model cannot retrieve it, whereas a graph walk finds it deterministically. A P&ID is *already* a graph; the system recovers that structure rather than flattening it into vectors.

3. The domain ontology is configuration, not code

The knowledge-graph schema lives in `config/ontology.yaml`. A `general` pack covers any corpus; an `industrial` pack adds refinery types. Packs merge, and a new domain is a YAML block rather than a code change. This was validated by indexing a corpus of curricula vitae and a journal article, which produced 499 nodes across Person, Concept, Technology, Publication and Skill types.

2 The Problem Statement

2.1 Official description

SIH26117 — Mangalore Refinery and Petrochemicals Limited

Background. Refineries, PSUs, defence-linked manufacturing units and government offices generate a lot of routine but sensitive knowledge work: approval notes, board presentations, engineering calculations, code for internal tools, review of scanned drawings and inspection reports. None of this can go through cloud AI assistants because the underlying data is confidential: piping and instrumentation diagrams, financials, vendor negotiations, unreleased designs, internal correspondence, confidential business strategies. Company policy keeps this data on premises, so people either do the work manually — losing productivity — or they quietly paste confidential material into public tools anyway.

2.2 Decomposition of the requirements

The statement contains five explicit demonstration requirements. Each is treated in this report as a verifiable acceptance criterion.

#	Requirement as stated	Section
1	Model auto-selection across at least two different task types	§9.1.1
2	An agentic task carried end to end, e.g. reading a scanned inspection report, extracting findings, drafting an approval note as a Word file	§9.1.2
3	A coding task run and verified in a sandbox	§9.1.3
4	A multimodal task involving image or scanned document understanding	§9.1.4
5	Evidence, through logs or a visible network monitor, that no external calls are made at any point	§8

Alongside these, the statement imposes four architectural constraints:

1. **Not locked to one model.** Multiple open-weight models must be supported simultaneously, with automatic selection based on task needs.
2. **Extensible without redesign.** New open-weight models must be addable later, because the field moves quickly.

3. **Genuinely agentic.** The system must plan multi-step work, call local tools, and iterate — not answer once and stop.

4. **Real deliverables.** Output must be approval notes, office documents, working code and calculations with steps shown — not chat replies.

2.3 Hardware constraint

The statement permits demonstration on "a single workstation or server with a mid-range GPU", explicitly allowing a smaller open-weight model if 120B-class hardware is unavailable at the venue. The reference implementation therefore targets a single 16 GB consumer card, and treats the ability to run on modest hardware as a feature rather than a compromise: an organisation that must procure an H100 before it can pilot the system will not pilot the system.

2.4 Why this is hard

Memory is the binding constraint.
A 16 GB card cannot hold a reasoning model, a coding model and a vision model concurrently. Naive multi-model serving thrashes or crashes.

Proving a negative.
Demonstrating that *no* external call occurred is qualitatively harder than demonstrating a feature works. It requires enforcement at a layer the application cannot bypass.

Retrieval over structure.
Industrial questions are frequently about dependency and impact. These are graph problems wearing the clothing of search problems, and conventional retrieval-augmented generation answers them badly.

Trust in output.
A system that fabricates a plausible approval note is worse than no system. Every claim must carry provenance back to a source page.

3 Solution Overview

3.1 Design principles

Enforce, do not request.
Isolation is implemented where the application cannot override it: kernel network namespaces and firewall rules, not configuration flags.

Configuration over code.
The model catalogue and the domain ontology are YAML documents. Adding a model or a domain changes no Python.

Embedded over networked.
The graph database and vector store run in-process. On an air-gapped deployment there is no listening port for anything to reach, and no service to orchestrate.

Degrade, never fail.
If the embedding model is absent, graph traversal still answers structural questions. If a tool errors, the failure is reported to the agent rather than injected into a deliverable.

Provenance everywhere.
Chunks, graph nodes and graph edges all carry document, page and region identifiers, so any statement can be traced back.

3.2 Capability map

Capability	Mechanism	Module
Multi-model serving	Declarative manifest, capability tags, priorities	`config/models.yaml`
Task routing	Two-stage: heuristics first, 4B classifier fallback	`app/router/classifier.py`
Memory management	Budgeted LRU residency with eviction	`app/router/manager.py`
Agentic execution	Plan → Execute → Critique loop	`app/agent/loop.py`
Tool layer	12 schema-declared tools with argument coercion	`app/tools/`
Sandboxing	Unprivileged user + network namespace	`app/tools/sandbox.py`
Document understanding	ONNX OCR on CPU, vision model on GPU	`app/tools/vision.py`
Knowledge graph	Typed extraction, canonicalisation, embedded store	`app/graphrag/`
Hybrid retrieval	Dense + BM25 fused by reciprocal rank	`app/retrieval/vector.py`
Deliverables	Template-filled Office documents	`app/tools/docgen.py`
Sovereignty	Egress monitor, canary, hash-chained audit	`app/sentinel/`

3.3 A worked example

The following illustrates the full path through the system.

1. An engineer uploads a scanned inspection report and asks: *"Which units are affected if P-101A is isolated, and draft an approval note recommending seal replacement."*
2. The **router** classifies the request. The presence of a document attachment and deliverable vocabulary routes planning to the 14B reasoning model.
3. The **agent** produces a plan: query the graph for P-101A's neighbourhood; compute the impact; generate an approval note.
4. **graph_query** traverses three hops from `Equipment:P-101A` and returns E-204 (one hop, direct feed), V-1201 (two hops, indirect) and TI-2043 (the instrument monitoring E-204).
5. The result is **rendered** into readable prose and substituted into the document-generation step through a `{{step1}}` placeholder.
6. **make_approval_note** writes a real `.docx` with headings, an approval signature table and a provenance footer.
7. Throughout, the **sentinel** panel shows `external_connections: 0`, and the audit chain records every prompt, model choice and tool call.

The critical detail is step 4. V-1201 is never mentioned in the same sentence as P-101A anywhere in the corpus. It is reachable only by following `P-101A -FEEDS-> E-204 -FEEDS-> V-1201`. No embedding model retrieves it; a graph walk finds it every time.

4 System Architecture

4.1 Layered view

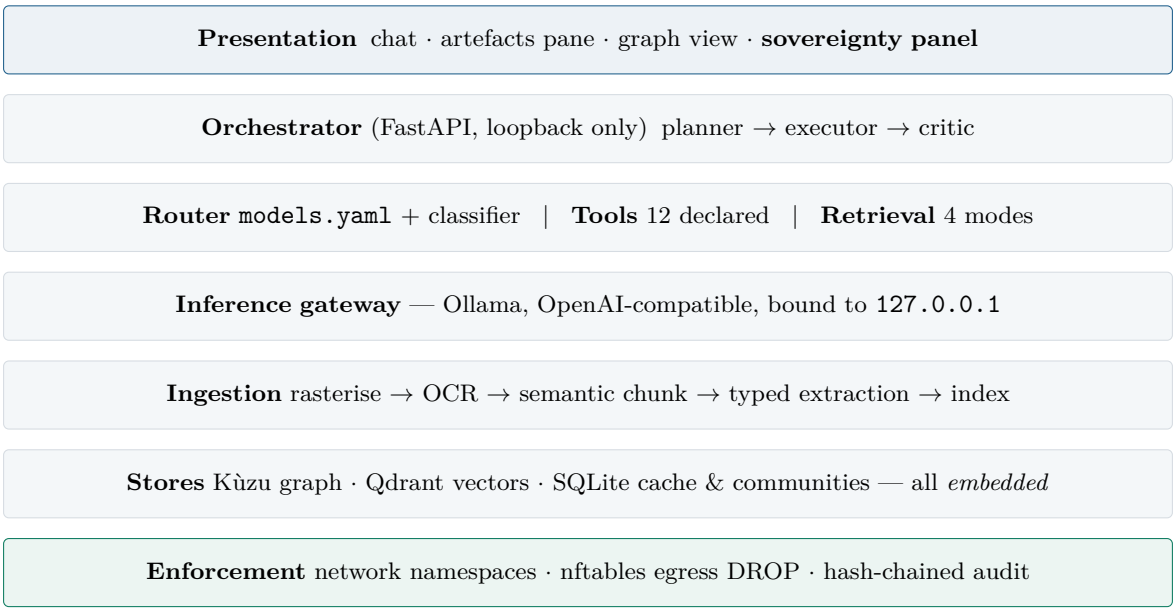

Figure 1: Seven layers. Each runs as a separate process or in-process library; none opens a port reachable from outside the host.

4.2 Why embedded stores

The original design used Neo4j and Qdrant as containers. Docker proved unavailable on the target machine without administrative rights, which forced a re-evaluation that produced a better result.

Need	Conventional	Chosen	Consequence
Graph database	Neo4j container	Kùzu (embedded)	no server, no port, no orchestration
Vector store	Qdrant server	Qdrant local mode	on-disk, in-process
Metadata	PostgreSQL	SQLite	zero configuration
Sandbox	Docker `--network none`	`unshare -rn`	unprivileged, kernel-enforced

For an air-gapped deployment this is strictly superior. A system with no listening sockets has no network attack surface to defend, and installation reduces to copying a directory.

4.3 Request lifecycle

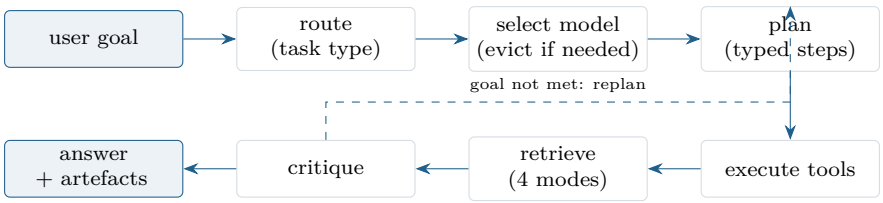

Figure 2: Every stage writes to the audit log. The critic may trigger up to n replanning iterations before synthesis.

4.4 Repository layout

```
sovereign-workbench/
  config/
    models.yaml model catalogue: capabilities, VRAM, priority
    ontology.yaml domain packs: node types, relations, tagging
  app/
    core/ config, Ollama client, hash-chained audit log
    router/ task classifier + VRAM-aware model manager
    agent/ plan-execute-critique loop, step data flow
    tools/ fs, sandbox, calc, docgen, vision, registry, render
    ingest/ PDF/OCR/semantic chunking with provenance
    graphrag/ ontology, extraction, canonicalisation, store, communities
    retrieval/ dense + sparse hybrid with reciprocal rank fusion
    sentinel/ egress monitor, canary, status
    main.py FastAPI application (loopback only)
  scripts/ 00 preflight, 01 pull, 02 index, 03 demo, 04 lockdown, 05 watch
  tests/ 61 tests
  ui/ single-page interface
  data/
    corpus/ source documents <- user input
    workspace/ agent scratch (jailed)
    artifacts/ generated deliverables
    stores/ graph, vectors, cache, audit
```

5 The Pipeline

The system contains two distinct pipelines: an *offline* indexing pipeline that runs when documents are added, and an *online* query pipeline that runs per request. Separating them matters because indexing is expensive and one-off, whereas queries must be fast.

5.1 Offline: indexing

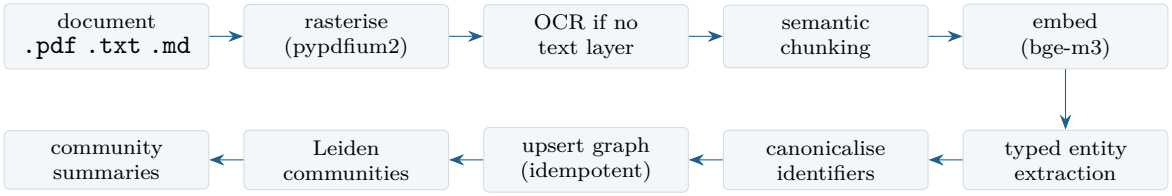

Figure 3: Indexing. Every chunk retains `{doc_id, page, bbox}`, so any downstream claim can be traced to a page region.

Rasterisation and OCR.
PDFs with a text layer are extracted directly. Pages without one are rendered at 2× and passed to an ONNX OCR engine on the CPU, keeping the GPU free for the language model.

Semantic chunking.
Splitting occurs on structure — headings, numbered clauses, blank lines — before falling back to size. Fixed-size chunking severs procedure steps from their headings, which is precisely the context the extractor needs.

Typed extraction.
Each chunk is passed to a small utility model with a JSON schema derived from the active ontology, using constrained decoding. Results are cached by `sha256(prompt + model)`.

Canonicalisation.
Identifiers are normalised so that every mention of an entity lands on one node. Seven spellings of an asset tag collapse to a single identity.

Community detection.
The graph is clustered with the Leiden algorithm and each cluster is summarised once, enabling corpus-wide thematic queries that no single passage could answer.

Caching makes re-indexing nearly free

The extraction cache is keyed on chunk content and model. Re-running the index over an unchanged corpus completes in seconds; adding one document costs only that document's chunks. The expensive first pass is paid once per document, not once per query.

5.2 Online: query

1. **Classify.** Heuristics resolve most requests at zero cost; a 4B model is consulted only when confidence is low.
2. **Select and load.** The manager checks residency, evicts least-recently-used models if the VRAM budget would be exceeded, and records the decision.
3. **Plan.** The reasoning model emits a typed list of steps with arguments, including `{{stepN}}` references to earlier results.
4. **Retrieve.** The query shape selects one of four strategies (§7.4).
5. **Execute.** Tool arguments are coerced to their declared schema, then invoked. Failures are captured, never injected into output.
6. **Critique.** The model assesses whether the goal was met and, if not, states what is missing; the loop may replan.
7. **Synthesise.** A grounded answer is produced from rendered tool results with citations, alongside any generated files.

6 Technology Stack

Each component below is given with a definition, a concrete example of its role in this system, and the reason it was selected over alternatives.

6.1 Model serving and inference

6.1.1 Ollama

Definition.
A local inference server that runs open-weight language models from quantised GGUF files and exposes an HTTP API. It loads models on demand and unloads them after an idle interval.

Example.
`POST /api/chat` with `{"model": "qwen3:14b", "format": <json-schema>}` returns output guaranteed to satisfy the schema.

Why chosen.
Automatic load and unload gives VRAM management essentially for free, which is decisive on a 16 GB card. It ships CUDA 12.8 builds that work on Blackwell today, and air-gapping is simply

copying `~/.ollama`. vLLM offers higher throughput but no automatic model swapping, which matters more here than tokens per second.

6.1.2 Open-weight models

Role	Model	VRAM	Why this one
Reasoning, planning	`qwen3:14b`	11.0 GB	strongest planner that fits with headroom
Routing, extraction	`qwen3:4b`	4.0 GB	thousands of calls; must be cheap
Coding	`qwen2.5-coder:7b`	6.0 GB	specialised for code generation
Vision	`qwen2.5vl:7b`	8.0 GB	reads scans, handwriting, drawings
Embeddings	`bge-m3`	1.2 GB	dense *and* sparse in one pass

Quantisation is 4-bit throughout. A `datacenter` profile substitutes `gpt-oss:20b` where more memory is available, addressing the problem statement's reference to 120B-class hardware.

6.2 Retrieval and storage

6.2.1 Kùzu — embedded property graph database

Definition.
An in-process graph database with a Cypher-like query language, analogous to what SQLite is for relational data.

Example.
`MATCH (a:Equipment {key:$k})-[r*1..3]-(b) RETURN b` returns everything within three hops of an asset.

Why chosen.
No server process and no listening port. On an air-gapped box this is a security property, not merely a convenience.

6.2.2 Qdrant (local mode) — vector store

Definition.
A similarity search engine over embedding vectors; local mode runs as an on-disk library rather than a service.

Example.
A 1024-dimensional `bge-m3` embedding is stored per chunk with its `doc_id` and page; cosine search returns nearest passages.

Why chosen.
Native support for combined dense and sparse vectors, and the ability to run embedded.

6.2.3 BM25 — lexical ranking

Definition.
A term-frequency ranking function that scores documents by exact word overlap, weighted by rarity.

Example.
The query `P-101A` matches the literal token; an embedding model may instead return semantically similar but wrong assets such as P-101B.

Why chosen.
Exact identifier matching is a hard requirement in industrial corpora, and it is precisely where dense retrieval is weakest.

6.2.4 Reciprocal Rank Fusion

Definition.
A method of combining ranked lists using $\text{score}(d) = \sum_i 1/(k + \text{rank}_i(d))$, where k is a smoothing constant, conventionally 60.

Example.
A passage ranked 1st by BM25 and 5th by vector search scores $1/61 + 1/65 = 0.0318$.

Why chosen.
It is scale-free. Cosine similarities and BM25 scores live on incomparable scales, and fusing by rank avoids ever calibrating them.

6.2.5 Leiden community detection

Definition.
A graph clustering algorithm that partitions a network into densely connected communities, improving on Louvain by guaranteeing well-connected clusters.

Example.
Applied to the indexed corpus it produced 37 communities, each summarised once so that "what are the recurring themes" can be answered.

Why chosen.
Thematic questions have no single source passage. Only pre-computed cluster summaries can answer them.

6.3 Document understanding

6.3.1 RapidOCR (ONNX Runtime)

Definition.
An optical character recognition pipeline — text detection, angle classification, recognition — executed through ONNX Runtime on the CPU.

Example.
A skewed, noisy scan of an inspection report was processed in 2.4 seconds, recovering every asset tag correctly.

Why chosen.
It requires no GPU and no PyTorch, so it leaves the entire card for the language model, and it installs cleanly offline.

6.3.2 Vision-language model

Definition.
A model accepting interleaved image and text input, able to reason about layout, symbols and handwriting rather than only transcribe.

Example.
Asked to list visible tags and the vibration reading in a scanned report, it returned P-101A, TI-2043, E-204 and "7.2 mm/s RMS (alarm 4.5, trip 9.0)".

Why chosen.
OCR transcribes; it does not comprehend. A P&ID requires understanding of symbols and connectivity, not character recognition.

6.4 Agent and tooling

6.4.1 Constrained decoding

Definition.
Restricting token generation so output provably conforms to a supplied JSON schema.

Example.
Entity extraction cannot emit a type outside the ontology enumeration, because such tokens are masked during sampling.

Why chosen.
It removes an entire class of parsing failure. Prompting a model to "reply in JSON" and then parsing prose is unreliable at scale.

6.4.2 Linux namespaces

Definition.
A kernel feature partitioning global resources so processes see isolated views. A network namespace provides a private stack with only a down loopback interface.

Example.
`unshare -rn python main.py` runs generated code where `socket.create_connection(("1.1.1.1", 53))` raises `OSError: Network is unreachable`.

Why chosen.
It requires no root, and the guarantee is enforced by the kernel rather than by application policy.

6.4.3 SymPy

Definition.
A symbolic mathematics library performing exact algebraic manipulation rather than floating-point approximation.

Example.
Net positive suction head, evaluated symbolically then substituted, returns 11.810537 with every intermediate step recorded.

Why chosen.
The problem statement requires "calculations with steps shown". Language models perform arithmetic unreliably and produce no auditable derivation; symbolic evaluation produces both.

6.4.4 python-docx, python-pptx, openpyxl

Definition.
Libraries constructing genuine Office Open XML documents.

Example.
An approval note is emitted with a title, numbered findings, a three-column signature table and a provenance footer.

Why chosen.
Filling templates from structured fields is far more reliable than asking a model to emit document XML, and the result resembles the organisation's own paperwork.

6.5 Application and platform

Component	Role and rationale
FastAPI + Uvicorn	Async HTTP API bound to `127.0.0.1`; no external interface
Pydantic	Typed settings and schema validation at every boundary
httpx	Single auditable HTTP transport, pinned to loopback by assertion
NetworkX + python-igraph	Graph construction and Leiden clustering
PyTorch (cu128)	CUDA runtime for Blackwell `sm_120`; used by ONNX paths
nftables	Kernel firewall implementing default-deny egress
SQLite	Extraction cache and community summaries; zero configuration

Verified platform

Python 3.11.15 in a project-local virtualenv; PyTorch 2.11.0+cu128 confirmed on compute capability $(12, 0)$; NVIDIA RTX 5080 16 GB; 32 CPU cores; 59 GB system RAM. Ninety-nine packages, all installed inside the virtualenv; nothing was installed system-wide.

7 Graph RAG

7.1 Why conventional RAG is insufficient

Standard retrieval-augmented generation embeds text chunks and retrieves those nearest a query. This works for lookup questions and fails for three classes of question that dominate industrial use.

Question	Requires	Vector search
What does the SOP say about pump lubrication?	lookup	✓ works
If we isolate P-101A, what is affected downstream?	traversal	× fails
Which vendors supplied valves in corrosion incidents since 2021?	join	× fails
What are the recurring failure themes across five years?	aggregation	× fails

The failure is structural, not a matter of tuning. Consider the corpus fragment `P-101A -FEEDS-> E-204` and `E-204 -FEEDS-> V-1201`. The query "what is affected if P-101A is isolated" must return V-1201. But V-1201 never appears in any sentence containing P-101A, so no embedding of the query is near any chunk mentioning V-1201. Similarity is not connectivity. Increasing k, changing the embedding model, or reranking cannot fix this; the information required is in the *topology*, which embeddings discard.

A P&ID is already a graph

The strongest argument for this approach in an industrial setting is that piping and instrumentation diagrams are graphs by construction: equipment are nodes, process lines are edges. Flattening them into embeddings destroys the structure the document was drawn to express. Extracting a graph does not impose a foreign abstraction — it recovers the native one.

7.2 The ontology

Constraining extraction to a closed schema is the single largest quality lever. Free-form extraction produces a graph in which "Pump 101A", "P-101A" and "the main feed pump" are three unrelated nodes, and every traversal returns a fragment of the truth.

The schema is a configuration artefact, selected by an environment variable:

```
packs:
  general: # domain-neutral: any corpus
    nodes:
      Person: [role, affiliation, email]
      Concept: [definition, domain]
      Technology: [category, vendor]
      Publication: [venue, year, doi]
      ...
    relations:
      AUTHORED_BY: [[Document, Person], [Publication, Person]]
      PART_OF: [[Section, Document], [Project, Organization]]

  industrial: # refinery / plant extension
    nodes:
      Equipment: [tag, equip_type, unit, description]
      Instrument: [tag, instr_type, measures, unit]
    tagged: [Equipment, Instrument, Line]
    relations:
      FEEDS: [[Equipment, Equipment]]
      PART_OF: [[Equipment, Equipment]]
```

Three properties make this robust:

Packs merge rather than overwrite.
`PART_OF` is declared in both packs. The merged ontology retains every endpoint signature — `Section`→`Document`, `Project`→`Organization` *and* `Equipment`→`Equipment` — implemented as one relation table carrying multiple `FROM ...TO` pairs.

Cross-pack references degrade gracefully.
The industrial pack references `Person`, which lives in the general pack. Enabling industrial alone drops those signatures instead of failing to start.

Tagging is opt-in.
Only types listed under `tagged:` pass through identifier canonicalisation. A corpus of research papers is never forced through asset-tag parsing.

7.3 Entity resolution

Identifier canonicalisation is where naive implementations quietly fail. The resolver normalises Unicode, removes stop words, maps spelled-out equipment nouns to their conventional function codes, and matches an ISA-5.1 shaped pattern anywhere in the string.

Surface form	Canonical key
P-101A, P101A, PUMP-101A	Equipment:P-101A
Pump 101-A, pump P101 A	Equipment:P-101A
the main pump P-101 A	Equipment:P-101A
centrifugal pump 101 A, Pump no. 101 A	Equipment:P-101A
Exchanger 204, E-204, heat exchanger E204	Equipment:E-204
Fire Water	(not a tag — name slug)

All seven spellings of the first asset collapse to one node. During development a defect in this component — a stop-word list that stripped a bare `A` — silently split `P-101A` from `P-101`, fragmenting the graph without any error. It is now covered by regression tests.

7.4 Retrieval modes

Query shape selects the strategy.

Mode	Trigger and mechanism	Answers
`vector`	no structural signal; BM25 + dense fused by RRF	lookup
`local`	an identifier is present; one-hop neighbourhood + linked chunks	context
`traverse`	impact vocabulary *and* an identifier; three-hop walk	dependency
`global`	corpus-wide vocabulary; map-reduce over community summaries	themes

Measured behaviour on the indexed corpus:

```
[traverse] If we isolate P-101A which downstream units are affected?
          -> nodes=7 chunks=3 (V-1201 reached at two hops)
[global ] what are the recurring failure themes?
          -> communities=2 consulted
[local ] what does the SOP say about P-101A
          -> nodes=5 chunks=3 (V-1201 NOT present: one hop only)
[vector ] what is the bearing vibration alarm limit
          -> chunks=3
```

The contrast between `traverse` and `local` is the demonstration: the same anchor entity, differing only in hop depth, and the indirect dependency appears only in the deeper walk.

7.5 Result on a general corpus

To validate that the ontology is genuinely domain-portable, a corpus containing three curricula vitae and a fifteen-page electronics journal article was indexed with `ONTOLOGY=general,industrial`.

Node type	Count	Relation	Count
Person	152	AUTHORED_BY	158
Concept	94	MENTIONS	83
Technology	68	HAS_SKILL	37
Skill	35	USES	30
Publication	28	APPLIED_TO	13
Document	26	REPORTS_METRIC	12
Section	17	COMPARED_WITH	10
Organization	14	AFFILIATED_WITH	10
others	65	others	38
Total	**499**	**Total**	**391**

Thirty-seven communities were detected and summarised. The industrial types remained near-empty, as expected for this corpus — confirming that the packs neither interfere with one another nor force inappropriate structure onto the data.

7.6 A defect worth recording

The first generation of community summaries stated relationships backwards: the graph asserted `V-4401 PART_OF P-101A` while the summary read "P-101A is a component of V-4401". The cause was that the clustering graph was built with an undirected `networkx.Graph`, so edge orientation was lost before the summarisation prompt was rendered. Clustering legitimately requires an undirected view, but rendering does not. The graph is now constructed as a `DiGraph` and converted to undirected only for the Leiden step. In a system whose central claim is grounded accuracy, confidently reversed relationships are among the most damaging failures possible, and they produced no error of any kind.

8 Sovereignty and Security

The problem statement is unusually direct about this requirement:

> *The system should also show, through logs or a visible network monitor, that no external calls are made at any point. That's the actual proof of the sovereign claim, not just a statement of it.*

Accordingly, sovereignty is implemented in four independent layers, so that no single failure silently removes the guarantee.

8.1 Layer 1 — kernel network isolation

Generated code executes inside an unprivileged user and network namespace created with `unshare -rn`. The child process receives a private network stack containing only a down loopback interface. Supplementary controls: RLIMIT caps on CPU, address space, process count and file size; a scrubbed environment with credential-shaped variables removed; a scratch working directory; and a wall-clock timeout.

```
>>> run_python("import socket; socket.create_connection(('1.1.1.1', 53))")
backend = unshare-netns
stdout = blocked: OSError [Errno 101] Network is unreachable
```

This is a kernel guarantee. The code is not prevented from calling out by policy or by a library wrapper; there is no route for the packet to take. Where Docker is available the sandbox upgrades automatically to `--network none` with a read-only root filesystem.

8.2 Layer 2 — host egress denial

At the venue, `scripts/04_lockdown.sh` installs an nftables table with a default `drop` policy on the output hook, permitting only loopback and established connections. Ollama, bound to `127.0.0.1`, continues to function; everything else stops.

```
table inet sovereign {
  chain output {
    type filter hook output priority 0; policy drop;
    oif "lo" accept
    ip daddr 127.0.0.0/8 accept
    ct state established,related accept
    counter comment "dropped-egress"
  }
}
```

8.3 Layer 3 — live monitoring and the canary

A background sentinel samples connection state and classifies every peer as local or external, exposing counters through `GET /sovereignty` and a persistent panel in the interface. A deliberate breach attempt is available at `POST /sovereignty/canary`.

```
{"blocked": true, "verdict": "BLOCKED",
 "detail": "TimeoutError: timed out", "ms": 3003}
```

The canary is the most valuable twenty seconds of any demonstration: it converts an assertion into an observation the audience can watch happen. The monitor reports honestly in both directions — on a development machine with ordinary internet access it correctly reports that the call succeeded.

8.4 Layer 4 — tamper-evident audit

Every prompt, model selection, tool invocation and file write is appended to a hash chain. Each record embeds the SHA-256 digest of its predecessor, so any modification or deletion breaks verification and the first corrupted index is reported.

```
record_n = { ts, event, payload, prev: hash(record_{n-1}) }
hash_n = sha256(canonical_json(record_n))
```

A regression test alters one field in the middle of a chain and asserts that verification fails at exactly that index. This makes the audit log suitable as compliance evidence rather than merely as a debugging aid.

8.5 Additional controls

Control	Implementation
Loopback assertion	The LLM client refuses to construct against a non-local host
No listening ports	Graph and vector stores are embedded libraries
Filesystem jail	All file tools resolve paths and reject workspace escape
Offline model cache	`HF_HUB_OFFLINE`, `TRANSFORMERS_OFFLINE` set
Typed tool arguments	Coerced and validated before invocation
No error leakage	Failed steps cannot be substituted into deliverables

8.6 Threat model

Threat	Status	Mitigation
Model exfiltrates data over the network	Mitigated	no route exists from the host
Generated code opens a socket	Mitigated	network namespace
Generated code reads outside the workspace	Mitigated	path jail; sandbox scratch dir
A dependency phones home on import	Mitigated	host egress denial; offline flags
Audit log altered after the fact	Detected	hash chain verification
Prompt injection from an ingested document	**Partial**	tools are typed and enumerated, but a malicious document could still steer the planner
Malicious insider with shell access	**Out of scope**	requires OS-level controls

9 Implementation and Verification

9.1 Rubric compliance

Each of the five demonstration requirements was executed and observed. The evidence below is drawn from actual runs, not from design intent.

9.1.1 Point 1 — model auto-selection

Routing is two-stage: regular-expression heuristics resolve the majority of requests at zero cost, and a 4B classifier with constrained output is consulted only when confidence falls below threshold. Selection then passes to a manager that enforces a VRAM budget with least-recently-used eviction.

```
resident before : ['reason'] 11.0GB / 13.5GB budget
vision model : qwen2.5vl:7b (25s)
resident after : ['vision'] 8.0GB

ROUTER DECISION LOG
  plan -> qwen3:14b highest-priority model with 'plan'
  vision -> qwen2.5vl:7b fits 8.0GB in 13.5GB evicted=['reason']
```

Two task types, two different models, and an observed eviction — the requirement met literally. Adding a model is a block in `config/models.yaml`, satisfying "addable later without redesigning the system".

9.1.2 Point 2 — agentic task end to end

```
GOAL determine which equipment is affected if P-101A is isolated,
     then produce an approval note as a docx

STEPS graph_query 22 ms identify connected equipment
      calculate 51 ms analyse isolation impact
      make_approval_note 13 ms generate formal note

ARTEFACT data/artifacts/seal_replacement_approval.docx
```

The generated document contains a title, subject block, numbered findings, a recommendation, a three-column approval signature table and a source-reference section — populated from graph output, not invented.

9.1.3 Point 3 — sandboxed code execution

Verified above (§8). Code runs, produces files, and cannot reach the network.

9.1.4 Point 4 — multimodal understanding

A synthetic scan with rotational skew and per-pixel noise was processed by both paths:

Path	Time	Outcome
RapidOCR (CPU)	2.4 s	all asset tags recovered; `SOP` read as `SoP`
Vision model (GPU)	25 s	tags plus the vibration reading, in context

The case error illustrates why canonicalisation matters: raw OCR output is never trustworthy as an identifier without normalisation.

9.1.5 Point 5 — sovereignty proof

Verified above: kernel isolation, egress denial, live monitor, canary and a verified audit chain.

9.2 Defects found by execution

Nine defects were discovered by running the system rather than by reading it. They are recorded here because several are non-obvious and all are now covered by tests.

#	Defect	Severity
1	Node tables redeclared the built-in `name` column, so schema creation failed	blocking
2	A stop-word list stripped a bare `A`, silently splitting `P-101A` from `P-101`	**silent corruption**
3	Relations used bare `CREATE`; re-indexing duplicated every edge	data integrity
4	Vector points were keyed by offset; re-indexing appended a second copy of the corpus	data integrity
5	Planner-supplied arguments failed type checks, wasting whole iterations	functional
6	Steps could not reference earlier results, so documents contained invented placeholders	functional
7	Raw Python dictionaries were substituted into document prose	output quality
8	A failed step's stack trace was written into an approval note's findings	**severe**
9	Community summaries stated relationships backwards	**silent corruption**

The pattern worth noting

Defects 2, 8 and 9 produced no error. The system reported success while emitting wrong output — a fragmented graph, a traceback presented as an engineering finding, and confidently reversed dependency claims. In a system whose value proposition is trustworthy grounding, silent wrongness is the failure mode that matters, and only execution reveals it.

9.3 Test suite

Sixty-one tests execute in under one second, covering identifier canonicalisation, extraction validation against a hostile model response, routing, retrieval mode selection, sandbox isolation, filesystem jail escape, audit chain tampering, ontology pack merging and every defect listed above.

10 Measured Performance

All figures below were obtained on the reference machine: NVIDIA RTX 5080 (16 GB, `sm_120`), 32 CPU cores, 59 GB RAM, NVMe storage.

10.1 Interactive operations

Operation	Latency	Notes
Retrieval, all four modes	20–60 ms	graph traversal and hybrid search
Graph neighbourhood, three hops	22 ms	Kùzu, embedded
Tool invocation (non-LLM)	1–60 ms	filesystem, calculation, document generation
OCR, one scanned page	2.4 s	CPU, ONNX, GPU untouched
Vision model, one page	25 s	includes model load and eviction
Model swap (evict + load)	5–10 s	measured during the vision transition
Full agent run, three steps	45–85 s	planning, execution, critique, synthesis
Test suite, 61 tests	0.95 s	no model calls

Interactive latency is dominated by language-model generation, as expected. Retrieval is never the bottleneck — a useful property, since it means richer retrieval can be afforded without hurting

responsiveness.

10.2 Indexing throughput and its limitation

Indexing the reference corpus — five documents, eighteen pages, 59 chunks — took four hours and five minutes, an average of **250 seconds per chunk**. This is unacceptably slow and the cause was diagnosed rather than guessed.

Hypothesis	Verdict	Evidence
GPU throttling	rejected	98% utilisation, 73 °C, 2850 MHz sustained
CPU offload	rejected	`/api/ps` shows 5.1 GB weights fully resident
Oversized chunks	rejected	uniform 1 072 characters mean
Generation volume	**confirmed**	4 759 characters of discarded reasoning per call

At approximately 70 tokens per second, 250 seconds implies on the order of 17 000 generated tokens per extraction call. Two causes are identified:

1. **Reasoning mode is enabled by default.** The utility model emits thousands of chain-of-thought characters before the answer. When output is schema-constrained JSON, this reasoning is discarded and buys nothing.
2. **Generation is unbounded.** No `num_predict` ceiling is set, so a dense chunk against a 23-type ontology can produce an arbitrarily long entity list.

Remediation applied and measured

Setting `think: false` and a `num_predict` ceiling per task, exposed as a `task_options` block in `config/models.yaml`, reduced extraction from 250 to **18.1 seconds per chunk** — a **13.8**× improvement. The reference corpus now indexes in roughly 18 minutes rather than four hours. A fourth cause was found alongside these: the agent loop ran a critique step even on its final permitted iteration, where the verdict cannot trigger a replan, discarding a full generation on every query.

10.3 Effect of the remediation

Measurement	Before	After	Factor
Single call, `qwen3:14b`	11.0 s	3.0 s	3.7×
Reasoning characters per call	2 069	0	—
LLM calls per query	4	3	—
Full agent run	45–121 s	**12.7 s**	≈5–9×
Extraction per chunk	250 s	**18.1 s**	**13.8**×
59-chunk corpus	4 h 05 m	≈18 min	13.6×

Stage breakdown of the optimised 12.7 s run: planning 7.1 s, tool execution 1.1 s, synthesis 4.6 s. Planning is now the dominant cost, and is the correct place to spend further effort.

10.4 Why this matters less than it appears

Indexing is a one-time cost per document, not a per-query cost.

Action	Cost
First index of a document	the measured rate above
Re-running the index over an unchanged corpus	seconds (all cache hits)
Adding one document to an indexed corpus	that document's chunks only
Answering a question	no extraction at all

The extraction cache is keyed on `sha256(prompt + model)` and persists across runs, so an interrupted build resumes rather than restarts.

10.5 Extraction yield

Quantity	Value	Derived
Chunks indexed	59	from 18 pages, ≈ 3.3 chunks per page
Mean chunk size	1 072 chars	semantic boundaries respected
Entity mentions extracted	602	10.2 per chunk
Unique nodes after resolution	499	17% merged by canonicalisation
Typed relations	391	6.6 per chunk
Communities detected	37	Leiden, minimum size 3

11 Enterprise Scaling and Sizing

This section addresses the question of what deploying the system at organisational scale actually requires. Figures marked *measured* derive from the reference implementation; those marked *projected* are extrapolations and are stated as such.

11.1 The sizing model

Three quantities determine every hardware decision.

$$\begin{aligned} \text{chunks} &= \text{pages} \times 3.3 && \text{(measured)} \\ \text{index bytes} &= \text{chunks} \times 12\,\text{KB} && \text{(5.4 KB vectors + 6.2 KB graph)} \\ \text{ingest GPU-hours} &= \frac{\text{chunks} \times t_{\text{chunk}}}{3600} && (t_{\text{chunk}} \approx 15\,\text{s optimised, projected}) \end{aligned}$$

Per-chunk storage decomposes as: a 1024-dimensional `float32` embedding (4 096 B), the chunk payload and metadata (≈1 300 B), approximately 8.5 graph nodes and 6.6 edges (≈6.2 KB). Source documents are additional and typically dominate raw storage while being irrelevant to query performance.

11.2 Worked storage examples

Corpus	Pages	Chunks	Index	Ingest (1 GPU)
Departmental archive	50 000	165 K	2.0 GB	≈29 days
Plant document set	500 000	1.65 M	20 GB	≈10 months
Refinery, all history	5 000 000	16.5 M	198 GB	impractical
Enterprise, multi-site	20 000 000	66 M	792 GB	impractical

Storage is cheap; ingestion is the real constraint

Index storage is modest even at twenty million pages — under a terabyte, well within a single NVMe array. The binding constraint is *GPU-hours for extraction*. Bulk ingestion at plant scale is a parallel batch problem requiring a dedicated ingestion cluster, not a bigger disk.

11.3 Mitigating the ingestion cost

Parallelism.
Extraction is embarrassingly parallel across chunks. N GPUs deliver close to $N\times$ throughput; 16 GPUs reduce the 500 000-page case from ten months to roughly nineteen days.

Continuous batching.
Replacing Ollama with vLLM for the extraction worker enables batched decoding, empirically worth a further 4–8× on short structured outputs.

Tiered extraction.
Not every document justifies graph extraction. Indexing everything for vector search while restricting graph extraction to governing documents — procedures, drawings, incident reports — typically reduces the graph workload by an order of magnitude.

Incremental ingestion.
After the initial load, only new and revised documents are processed. Steady-state volume is a small fraction of the bulk load.

11.4 Deployment tiers

Tier	GPU	RAM	Storage	Users	Corpus
Pilot	1 × 16–24 GB	64 GB	2 TB NVMe	5–15	≤50 K pages
Departmental	1 × 48 GB	128 GB	8 TB NVMe	25–75	≤500 K pages
Plant-wide	2–4 × 80 GB	256–512 GB	30 TB NVMe	100–300	≤5 M pages
Enterprise	8–16 × 80 GB	1 TB+	100 TB+	1 000+	20 M+ pages

Tier	Characteristics
Pilot	Exactly the reference implementation. One model resident at a time with eviction. Single workstation under a desk; no data-centre dependency. Proves value before procurement.
Departmental	A single 48 GB card holds the reasoning and vision models concurrently, removing swap latency. Supports the full `venue` profile without eviction.
Plant-wide	80 GB cards enable 120B-class open-weight models, matching the problem statement's `datacenter` profile. One GPU is dedicated to batch ingestion; the remainder serve interactive load. Graph and vector stores migrate from embedded to clustered.
Enterprise	Separate ingestion and serving clusters, high availability, per-site corpora with federated retrieval, role-based access control over graph partitions.

11.5 Concurrency

Interactive capacity is governed by key-value cache memory, not by model weights:

$$\text{concurrent sessions} \approx \frac{V_{\text{GPU}} - V_{\text{weights}}}{V_{\text{KV per session}}}$$

For a 14B model at 4-bit (≈9 GB) on an 80 GB card with roughly 1 GB of KV cache per 8 K-token session, the arithmetic gives about 70 sessions; 40–50 is a realistic planning figure once fragmentation and headroom are accounted for. Because agent runs are bursty rather than sustained, a served-user multiple of five to ten times the concurrent-session count is typical.

11.6 Architectural changes required above the pilot tier

The reference implementation makes deliberate choices that are correct for a single workstation and must change at scale.

Component	Pilot	Scale-out replacement
Graph store	Kùzu (embedded)	Neo4j cluster or Kùzu with a read-replica fan-out
Vector store	Qdrant local	Qdrant or Milvus cluster with sharding
Inference	Ollama	vLLM with continuous batching and tensor parallelism
Cache	SQLite	PostgreSQL or Redis
Sandbox	`unshare`	gVisor or Kata Containers under an orchestrator
Identity	none	OIDC with role-based graph partition access
Ingestion	in-process	a queue with distributed workers

A caution on the single-writer constraint

Kùzu permits one writing process. In the reference implementation this means indexing and serving cannot run concurrently — an acceptable limitation for a pilot and an unacceptable one for production. It is the first component that must be replaced when moving beyond a single workstation.

12 Deployment and Operation

12.1 Installation

The entire Python environment is project-local. Nothing is installed system-wide.

```
python3.11 -m venv .venv
./.venv/bin/pip install torch --index-url https://download.pytorch.org/whl/cu128
./.venv/bin/pip install -r requirements.txt
./scripts/01_pull_models.sh # approximately 24 GB of weights
```

The CUDA 12.8 index is mandatory on Blackwell: earlier PyTorch binaries lack `sm_120` kernels and fail at runtime with `no kernel image is available for execution`.

12.2 Operating commands

Command	Purpose
`make check`	Preflight: GPU, models, sandbox capability, binaries
`make index`	Ingest `data/corpus` into vectors, graph and communities
`make watch`	Live indexing dashboard with rate and estimated completion
`make serve`	API and interface on `127.0.0.1:8077`
`make demo`	Scripted run covering all five rubric points
`make test`	61 tests
`make lockdown`	Install nftables egress denial (requires root)

12.3 Document placement

Directory	Contents
`data/corpus/`	**Source documents.** `.pdf`, `.txt`, `.md`; subdirectories are traversed
`data/workspace/`	Agent scratch space; file tools are jailed here
`data/artifacts/`	Generated deliverables
`data/stores/`	Graph, vectors, extraction cache, audit log

The filename becomes the `doc_id` used in every citation, so documents should be named with their real identifiers. Images (`.png`, `.jpg`, `.tiff`) are *not* automatically indexed — they are readable on demand through the vision tools, or may be converted to PDF for indexing.

12.4 Air-gap procedure

```
tar czf models.tar.gz -C ~/.ollama models # model weights
./.venv/bin/pip download -r requirements.txt -d ./wheels
export HF_HUB_OFFLINE=1 TRANSFORMERS_OFFLINE=1
sudo ./scripts/04_lockdown.sh on # egress denial
```

The offline rehearsal should be performed in the first week of a deployment, not the last. A dependency that resolves a hostname on import is common and is only discovered by testing with the interface down.

12.5 Operational cautions

Single writer.
Indexing and serving cannot run concurrently; the graph store holds an exclusive lock. Stop the

server before re-indexing.

Schema changes require a rebuild.
Altering the enabled ontology packs changes the database schema. Delete `data/stores/graph` and re-index.

The canary reports honestly.
On a machine with ordinary internet access it will correctly report that an external call succeeded. That is the monitor working, not failing. Apply the lockdown script to see it blocked.

13 Limitations and Risks

13.1 Known limitations

Area	Limitation	Severity
Ingestion rate	reduced 250 s → 18.1 s per chunk; still the slowest stage at bulk scale	medium
Concurrency	Embedded graph store permits one writer	high at scale
Image indexing	Standalone images are not auto-indexed, only read on demand	medium
Office formats	`.docx` and `.xlsx` sources are not ingested	medium
P&ID symbols	No trained symbol detector; the vision model is the only path	medium
Prompt injection	An ingested document could steer the planner	medium
Extraction recall	Bounded by a 4B model's competence on dense text	medium
Graph visualisation	Not implemented in the interface	low
Reranking	Fusion is by rank only; no cross-encoder stage	low

13.2 Risk register

Risk	Likelihood	Impact	Mitigation
Extraction quality degrades on unfamiliar domains	medium	high	ontology packs are user-editable; extraction is cached and re-runnable
Bulk ingestion exceeds the available window	high	high	tiered extraction; parallel workers; vector-only fallback
Silent output errors reach a signed document	low	**severe**	failed steps cannot be substituted; provenance on every claim; human approval block retained in every generated note
Venue hardware differs from development	medium	medium	profile switch in `models.yaml`; preflight script
A dependency attempts network access	medium	high	host egress denial; offline environment flags
Model licensing constraints	low	medium	all models are open-weight; the manifest permits substitution

13.3 An honest statement of maturity

This is a working reference implementation validated end to end on a single workstation, not a production system. Every one of the five demonstration requirements has been executed and observed rather than merely designed. The components most likely to require replacement before production use are, in order: the embedded graph store, the extraction throughput path, and the absence of any authentication or authorisation layer.

14 Roadmap

14.1 Immediate — correctness and speed

1. Disable reasoning mode and impose a generation ceiling on extraction and classification tasks. Expected to reduce indexing time by more than an order of magnitude.
2. Index standalone image files so that drawings placed in the corpus are not silently skipped.
3. Ingest `.docx` and `.xlsx` sources, which are ubiquitous in the target environment.
4. Improve the synthesis prompt so that generated prose reads naturally around substituted tool output.

14.2 Near term — demonstration quality

1. Graph visualisation in the interface, rendering the retrieved subgraph and animating traversal. This is the highest-value remaining item for a live demonstration.
2. A side-by-side comparison view showing vector-only retrieval failing the same dependency question that graph traversal answers.
3. A cross-encoder reranking stage after fusion.
4. Exercise the coding model with a goal that requires sandboxed execution, completing the routing demonstration across three distinct models.

14.3 Medium term — capability

1. A trained P&ID symbol detector, replacing sole reliance on the vision model for drawing comprehension.
2. Temporal reasoning over document revisions, so that superseded procedures are excluded from retrieval by default.
3. Confidence scoring on extracted relations, with a human review queue for low-confidence assertions.
4. Multi-document synthesis with conflict detection, flagging where two sources disagree rather than silently preferring one.

14.4 Long term — production

1. Replace embedded stores with clustered equivalents; separate the ingestion and serving planes.
2. Authentication, authorisation, and graph partitioning by clearance level.
3. Migrate the sandbox to gVisor or Kata Containers under an orchestrator.

4. Formal validation of the sovereignty guarantee, suitable for audit by an organisation's own information security function.

A Appendices

A.1 HTTP API

Method	Path	Purpose
GET	`/`	Single-page interface
GET	`/health`	Profile, graph statistics, tool count
POST	`/ask`	Run an agentic goal end to end
GET	`/search?q=`	Retrieval with chosen mode and rationale
GET	`/models`	Catalogue, residency, routing decision log
GET	`/sovereignty`	Egress counters, sandbox capability, audit validity
POST	`/sovereignty/canary`	Deliberate external call attempt
GET	`/audit`	Hash-chained audit tail with verification
GET	`/artifacts/{name}`	Download a generated deliverable

A.2 Tool catalogue

Tool	Function
`read_file`, `write_file`, `list_files`	Workspace-jailed filesystem access
`run_python`	Execution in a network-isolated sandbox
`calculate`	Symbolic evaluation returning the full derivation
`kb_search`	Retrieval with automatic mode selection
`graph_query`	Entity lookup and n-hop neighbourhood traversal
`ocr_document`	Exact text extraction from scans, CPU
`describe_document`	Vision-model comprehension of drawings and handwriting
`make_approval_note`	Formal `.docx` with approval block and provenance
`make_deck`	`.pptx` generation
`make_sheet`	`.xlsx` generation

A.3 Model manifest

```yaml
profiles:
  venue: # single 16 GB consumer card
    vram_budget_gb: 13.5
    models: [reason, code, vision, utility, embed]
  datacenter: # 120B-class hardware
    vram_budget_gb: 80
    models: [reason_xl, code, vision, utility, embed]

models:
  - id: reason
    tag: qwen3:14b
    capabilities: [reason, plan, doc, summarize]
    vram_gb: 11.0
    ctx: 32768
```

```
    priority: 10
```

A.4 Verified environment

Component	Version
Operating system	Ubuntu 26.04 LTS, kernel 7.0.0
Python	3.11.15 (project-local virtualenv)
PyTorch	2.11.0+cu128, compute capability $(12, 0)$ verified
GPU	NVIDIA RTX 5080, 16 GB, driver 595.84
CPU / RAM	32 cores / 59 GB
Ollama	0.33.3
Packages	99, all within the virtualenv

A.5 Glossary

Agentic
A system that plans multiple steps, invokes tools, observes results and iterates, rather than producing a single response.

Air-gapped
Physically or logically disconnected from external networks.

BM25
A lexical ranking function scoring exact term overlap weighted by rarity.

Chunk
A contiguous passage of a document treated as one retrieval unit.

Constrained decoding
Restricting token sampling so output provably conforms to a schema.

Embedding
A dense numeric vector representing meaning, such that similar texts lie close together.

Graph RAG
Retrieval augmented by a knowledge graph, enabling traversal and aggregation queries that similarity search cannot express.

KV cache
Stored attention keys and values for generated tokens; the dominant per-session memory cost.

Leiden
A community detection algorithm partitioning a graph into densely connected clusters.

Namespace
A kernel facility isolating a process's view of a global resource such as the network stack.

Ontology
The set of permitted entity and relation types constraining graph extraction.

Open-weight
A model whose parameters are published and may be run locally.

Provenance
The record of which document, page and region a claim originated from.

Quantisation
Representing weights at reduced precision to lower memory use.

RRF
Reciprocal Rank Fusion; combining ranked lists by rank rather than by score.

Sovereignty
The property that data and computation never leave organisation-controlled infrastructure.

A.6 Reproducing the results

```
git clone <repository> && cd sovereign-workbench
python3.11 -m venv .venv && source .venv/bin/activate
pip install torch --index-url https://download.pytorch.org/whl/cu128
pip install -r requirements.txt
./scripts/01_pull_models.sh
cp your-documents/* data/corpus/
make check && make index && make serve
```